



I. DATOS INFORMATIVOS.

Integrantes:	-Castelo Berrones Katherine Nicol -Chiluiza Quishpe Diego Steed -López Sánchez Washington Steven -Pilco Flores Mario David -Viteri Camino Maydelin Shantal	Carrera:	Software
Semestre:	Primer	Paralelo:	B
Asignatura:	Algoritmos y Lógica de Programación	Fecha:	05/05/2026
Docente:	Ing. José Caiza. Mg.		
Tema:	Implementación de estructuras básicas de datos		

II. DESARROLLO

Objetivo

Desarrollar un programa en lenguaje C++ y Java que permita ingresar dos arreglos de datos, identificar los elementos comunes entre ambos (intersección) sin repetir valores, y mostrar el resultado al usuario, validando además el caso en el que no exista ninguna coincidencia entre los arreglos.

• Ejercicio: Intersección sin repetidos

Dado dos arreglos A y B:

- Encuentra los elementos que están en ambos arreglos
- No repitas elementos en el resultado
- No utilices estructuras como conjuntos o mapas

LINK EN GITHUB: [Estructura-de-datos-y-Arreglos/T2.3 Implementación de estructuras basicas de datos at main · katth-i/Estructura-de-datos-y-Arreglos](#)

1. ANALISIS

Requerimiento:

Se tienen dos arreglos de números enteros, A y B.

El objetivo es encontrar los elementos que están presentes en ambos arreglos, es decir, su intersección.

Sin embargo, existen restricciones importantes:

No se deben repetir elementos en el resultado

No se pueden utilizar estructuras avanzadas como conjuntos o mapas

Por lo tanto, se debe resolver utilizando únicamente arreglos y lógica de control.

Variables de Entrada

Son los datos que el usuario debe proporcionar para que el programa funcione.

A: Arreglo de números enteros.

B: Arreglo de números enteros.

n: Tamaño del arreglo A.

m: Tamaño del arreglo B.

Procesos

Lectura de datos: Ingreso de los tamaños y elementos de los arreglos A y B.

Recorrido: Análisis de cada elemento del arreglo A.

Verificación de pertenencia: Comprobación de si un elemento de A existe en el arreglo B.

Verificación de duplicados: Validación de que el elemento no se encuentre previamente en el resultado.

Almacenamiento: Inserción de los elementos válidos en el arreglo resultado.



Presentación: Mostrar los elementos que pertenecen a la intersección.

Variables de Salida

Intersección: Arreglo que contiene los elementos comunes entre A y B sin repetir.

Mensaje de salida: Texto que indica el resultado obtenido.

2. ALGORITMO

Inicio

1. Leer n (tamaño del arreglo A)
2. Crear arreglo A de tamaño n
3. Para i = 0 hasta n-1 hacer
 Leer A[i]
FinPara
4. Leer m (tamaño del arreglo B)
5. Crear arreglo B de tamaño m
6. Para i = 0 hasta m-1 hacer
 Leer B[i]
FinPara
7. Crear arreglo resultado de tamaño n
8. Inicializar k = 0
9. Para i = 0 hasta n-1 hacer
 - a. enB ← falso
 - b. Para j = 0 hasta m-1 hacer
 Si A[i] == B[j] entonces
 enB ← verdadero
 Salir del ciclo
 FinSi
FinPara
 - c. repetido ← falso
 - d. Para r = 0 hasta k-1 hacer
 Si resultado[r] == A[i] entonces
 repetido ← verdadero
 Salir del ciclo
 FinSi
FinPara
 - e. Si (enB == verdadero) Y (repetido == falso) entonces
 resultado[k] ← A[i]
 k ← k + 1



FinSi

FinPara

10. Si $k == 0$ entonces

 Escribir "No hay interseccion entre los arreglos"

 Sino

 Escribir "Interseccion:"

 Para $i = 0$ hasta $k-1$ hacer

 Escribir resultado[i]

 FinPara

 FinSi

Fin

3. PSEUDOCODIGO

Algoritmo Interseccion_Arreglos

 Definir n, m, i, j, r, k Como Entero

 Definir enB, repetido Como Logico

 Escribir "Ingrese el tamaño del arreglo A: "

 Leer n

 Dimension $A[n]$

 Escribir "Ingrese los elementos de A:"

 Para $i <- 0$ Hasta $n-1$ Hacer

 Leer $A[i]$

 FinPara

 Escribir "Ingrese el tamaño del arreglo B: "

 Leer m

 Dimension $B[m]$

 Escribir "Ingrese los elementos de B:"

 Para $i <- 0$ Hasta $m-1$ Hacer

 Leer $B[i]$

 FinPara

 Dimension resultado[n]

$k <- 0$

 Para $i <- 0$ Hasta $n-1$ Hacer

 enB $<-$ Falso

 Para $j <- 0$ Hasta $m-1$ Hacer

 Si $A[i] = B[j]$ Entonces

 enB $<-$ Verdadero

 FinSi

 FinPara

 repetido $<-$ Falso



```
Para r <- 0 Hasta k-1 Hacer  
  Si resultado[r] = A[i] Entonces  
    repetido <- Verdadero  
  FinSi  
FinPara
```

```
Si enB = Verdadero Y repetido = Falso Entonces  
  resultado[k] <- A[i]  
  k <- k + 1  
FinSi
```

```
FinPara
```

```
Escribir "Interseccion: "
```

```
Para i <- 0 Hasta k-1 Hacer  
  Escribir Sin Saltar resultado[i], " "  
FinPara
```

```
FinProceso
```

4. DIAGRAMA DE FLUJO:

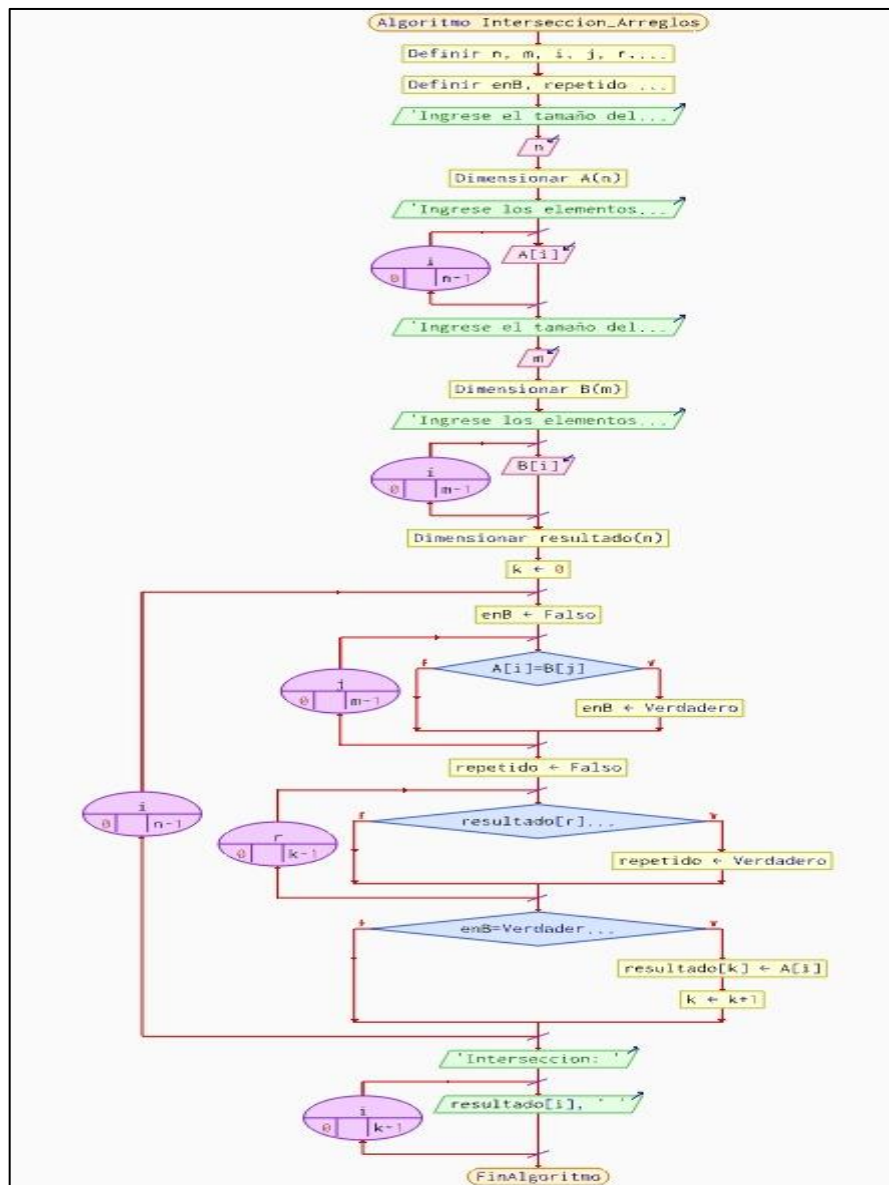


Imagen 1: Diagrama de flujo
Elaborado por: Grupo D



5. CÓDIGO EN:

C++

```
#include <iostream>
using namespace std;

int main() {
    int n, m;

    cout << "Ingrese el tamaño del arreglo A: ";
    cin >> n;
    int A[n];

    cout << "Ingrese los elementos de A:\n";
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    cout << "Ingrese el tamaño del arreglo B: ";
    cin >> m;
    int B[m];

    cout << "Ingrese los elementos de B:\n";
    for (int i = 0; i < m; i++) {
        cin >> B[i];
    }

    int resultado[n];
    int k = 0;

    for (int i = 0; i < n; i++) {
        bool enB = false;

        for (int j = 0; j < m; j++) {
            if (A[i] == B[j]) {
                enB = true;
                break;
            }
        }

        bool repetido = false;
        for (int r = 0; r < k; r++) {
            if (resultado[r] == A[i]) {
                repetido = true;
                break;
            }
        }

        if (enB && !repetido) {
            resultado[k] = A[i];
            k++;
        }
    }
}
```



```
if (k == 0) {  
    cout << "No hay interseccion entre los arreglos."  
} else {  
    cout << "Interseccion: "  
    for (int i = 0; i < k; i++) {  
        cout << resultado[i] << " "  
    }  
}  
  
return 0;  
}
```

JAVA

```
import java.util.Scanner;  
  
public class Interseccion {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Ingrese el tamaño del arreglo A: ");  
        int n = sc.nextInt();  
        int[] A = new int[n];  
  
        System.out.println("Ingrese los elementos de A:");  
        for (int i = 0; i < n; i++) {  
            A[i] = sc.nextInt();  
        }  
  
        System.out.print("Ingrese el tamaño del arreglo B: ");  
        int m = sc.nextInt();  
        int[] B = new int[m];  
  
        System.out.println("Ingrese los elementos de B:");  
        for (int i = 0; i < m; i++) {  
            B[i] = sc.nextInt();  
        }  
  
        int[] resultado = new int[n];  
        int k = 0;  
  
        for (int i = 0; i < n; i++) {  
            boolean enB = false;  
  
            for (int j = 0; j < m; j++) {  
                if (A[i] == B[j]) {  
                    enB = true;  
                    break;  
                }  
            }  
  
            if (enB) {  
                resultado[k] = A[i];  
                k++;  
            }  
        }  
  
        boolean repetido = false;  
        for (int r = 0; r < k; r++) {
```



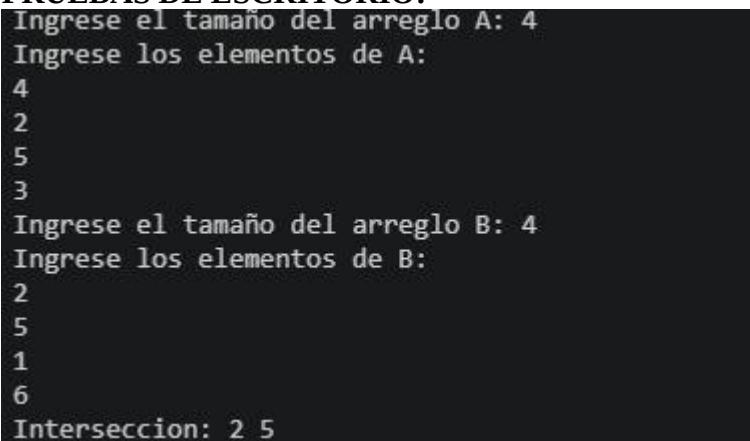
```
        if (resultado[r] == A[i]) {
            repetido = true;
            break;
        }
    }

    if (enB && !repetido) {
        resultado[k] = A[i];
        k++;
    }
}

if (k == 0) {
    System.out.println("No hay interseccion entre los arreglos.");
} else {
    System.out.print("Interseccion: ");
    for (int i = 0; i < k; i++) {
        System.out.print(resultado[i] + " ");
    }
}

sc.close();
}
```

6. PRUEBAS DE ESCRITORIO:



```
Ingrese el tamaño del arreglo A: 4
Ingrese los elementos de A:
4
2
5
3
Ingrese el tamaño del arreglo B: 4
Ingrese los elementos de B:
2
5
1
6
Interseccion: 2 5
```

Imagen 2: Prueba de escritorio #1

Elaborado por: Grupo D



```
Ingrese el tamaño del arreglo A: 6
Ingrese los elementos de A:
3
4
2
5
1
8
Ingrese el tamaño del arreglo B: 6
Ingrese los elementos de B:
3
4
23
5
6
2
Interseccion: 3 4 2 5
```

Imagen 3: Prueba de escritorio #2

Elaborado por: Grupo D

```
Ingrese el tamaño del arreglo A: 4
Ingrese los elementos de A:
1
2
3
4
Ingrese el tamaño del arreglo B: 4
Ingrese los elementos de B:
5
6
7
8
No hay interseccion entre los arreglos.
```

Imagen 4: Prueba de escritorio #3(Sin intersección entre los arreglos)

Elaborado por: Grupo D

7. RESULTADOS OBTENIDOS:

El programa permitió ingresar dos arreglos y determinó correctamente los elementos en común entre ambos, evitando repeticiones en el resultado. Se verificó su funcionamiento en distintos casos, incluyendo arreglos con valores duplicados y sin coincidencias. Además, cuando no existe intersección, el sistema muestra un mensaje informativo, garantizando una salida clara y correcta para el usuario.

8. CONCLUSIONES:

- El desarrollo del programa permitió aplicar correctamente estructuras básicas de programación para resolver el problema de la intersección de arreglos, logrando identificar elementos comunes sin utilizar estructuras avanzadas y evitando duplicados en el resultado.
- La implementación incluyó validaciones importantes, como la detección de ausencia de intersección, lo que mejora la claridad de la salida y demuestra la importancia de considerar todos los posibles escenarios al diseñar soluciones algorítmicas.

9. ANEXOS:



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE

Cda. Universitaria (Predios Huachi) / Casilla 334 / Telefax: 03-2851894 – 2411537, Correo Electrónico:
AMBATO-ECUADOR

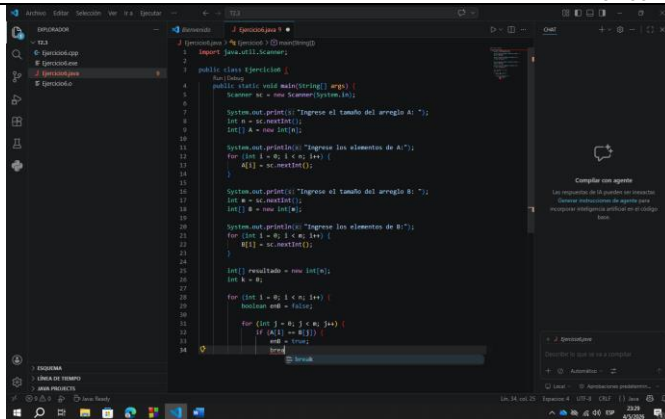


Imagen 5: Anexo#1

Elaborado por: Grupo D

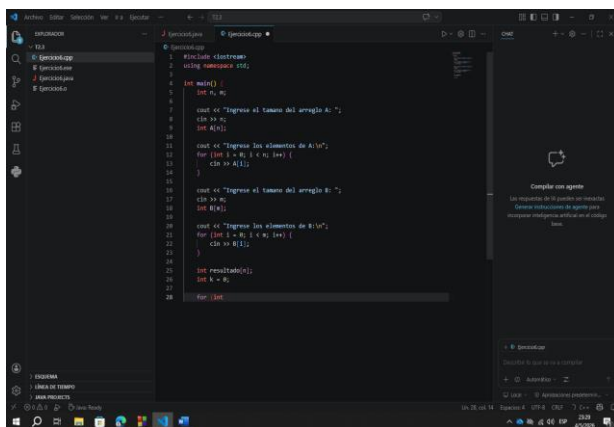


Imagen 6: Anexo #2

Elaborado por: Grupo D

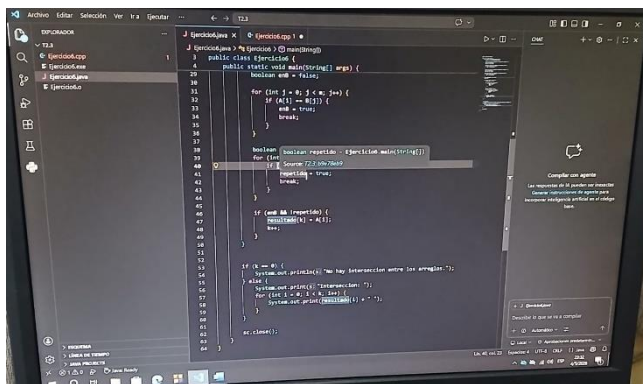


Imagen 7: Anexo #3

Elaborado por: Grupo D